B.Tech. (Artificial Intelligence & Mathematical Innovations)

SEMESTER-V

S. No.	Course No.	Course Title	Credits	Marks			Hours/Week			Total Marks
				100		50				
				Mid Semester	End Semester Exam	Project Cumulative	L	P/EL	T	
1	UMJDTT-501	Understanding Computer System Architecture & Circuits	6	30	70	50	4	2	0	150
2	UMJDTT-502	The Logic Behind Machines: Theory of Computation	6	30	70	50	4	2	0	150
3	UMJDTT-503	Information exchange in computing devices: Data Communication & Computer Networks	6	30	70	50	4	2	0	150
4	UMJDTT-504	Instructing computing devices: Operating System	6	30	70	50	4	2	0	150
5	UMJDTT-505	Building Blocks of Computing: Groups, Graphs, and Trees	6	30	70	50	4	2	0	150

***L- Lecture, *T- Tutorial, *P/EL- Practical / Experiential Learning**

B.Tech. (Artificial Intelligence & Mathematical Innovations)

SEMESTER-VI

S. No.	Course No.	Course Title	Credits	Marks			Hours/Week			Total Marks
				100		50				
				Mid Semester	End Semester Exam	Project Cumulative	L	P/EL	T	
1	UMJDTT-601	Linear Logic: The Art of Optimization	6	30	70	50	4	2	0	150
2	UMJDTT-602	Exploring Database Management Systems	6	30	70	50	4	2	0	150
3	UMJDTT-603	Architecting the Web: Modern Web Application Engineering	6	30	70	50	4	2	0	150
4	UMJDTT-604	Computer Graphics and Visualization using Java	6	30	70	50	4	2	0	150
5	UMJDTT-605	Data Structure Design using Object Oriented Programming (C/C++)	6	30	70	50	4	2	0	150

***L- Lecture, *T- Tutorial, *P/EL- Practical / Experiential Learning**

B.Tech. (Artificial Intelligence & Mathematical Innovations)

SEMESTER-VII

S. No.	Course No.	Course Title	Credits	Marks			Hours/Week			Total Marks
				100		50				
				Mid Semester	End Semester Exam	Project Cumulative	L	P/EL	T	
1	UMJDTT-701	Computer and Brain: Knowledge Discovery and Artificial Intelligence with Python / MATLAB	6	30	70	50	4	2	0	150
2	UMJDTT-702	Elective	6	30	70	50	4	2	0	150
3	UMJDTT-703	Robotics Process Automation & Drones	6	30	70	50	4	2	0	150
4	UMJDTA-704	AI in Neuro & Cognitive Computing	6	30	70	50	4	2	0	150
5	UMJDTA-705	Numerical methods: from approximation to perfection	6	30	70	50	4	2	0	150

***L- Lecture, *T- Tutorial, *P/EL- Practical / Experiential Learning**

Elective:
1- Quantum Computation
2- Crypto Currency & Block Chain Technologies
3- 3D Printing & Design
4- Cloud Computing
5- Cyber Security & Cyber Forensics

B.Tech. (Artificial Intelligence & Mathematical Innovations)

SEMESTER-VIII

S. No.	Course No.	Course Title	Credits	Marks			Houre/Week			Total Marks
				100		50				
				Mid Semester	End Semester Exam	Project Cumulative	L	P/EL	T	
1	UMJDTA-801	Advance, Generative AI & Agents	6	30	70	50	4	2	0	150
2	UMJDTT-802	The Science of Chance	6	30	70	50	4	2	0	150
3	UMJDTT-803	Computer Vision	6	30	70	50	4	2	0	150
4	UMJDTT-804	IoT Sensor Networks & Data Analytics	6	30	70	50	4	2	0	150
5	UMJDTT-805	Major Project / Industrial Internship (AI/ML/PYTHON/MATLAB)	6	30	70	50	4	2	0	150

***L- Lecture, *T- Tutorial, *P/EL- Practical / Experiential Learning**

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester V
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-501

Course Title: Understanding Computer System Architecture & Circuits

Credits: 06

Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

By the end of this course, students will be able to:

1. Understand and explain the principles, components, and performance metrics of computer architecture, including CPU, memory, I/O systems, and instruction set design.
2. Design and analyze digital circuits, microprocessor-based systems, and memory hierarchies using simulation tools and assembly programming.
3. Evaluate and apply advanced processor features such as cache mapping, branch prediction, parallel architectures, and system interfacing techniques.
4. Integrate theoretical concepts with practical applications through hands-on projects and simulations to solve real-world computing problems.

MODULE 1:

History and evolution of computer systems. Overview of architectures: Von Neumann vs. Harvard. Basic components: CPU, memory, I/O systems, system buses. Instruction Set Architecture (ISA): RISC vs. CISC, instruction types, addressing modes. Performance metrics: MIPS, CPI, FLOPS, Amdahl's Law.

Hands-On Activities:

1. Identify and compare architectures of different real-world processors.
2. Measure CPU performance metrics using benchmarking tools.

MODULE 2:

Boolean algebra and logic gates. Combinational circuits: adders, subtractors, multiplexers, encoders, decoders. Sequential circuits: flip-flops, registers, counters. Logic families: TTL, CMOS characteristics, power dissipation. Design and simulation of arithmetic and control circuits.

Hands-On Activities:

1. Simulate combinational and sequential circuits using software like Logisim or Multisim.
2. Design and test arithmetic units (e.g., 4-bit adder, ALU).

MODULE 3:

Internal architecture and pin configuration. Address, data, and control buses. Instruction set of 8085. Basics of Assembly language programming. Stack, subroutines, and interrupts.

Hands-On Activities:

1. Write and execute basic assembly programs for 8085 using an emulator.
2. Interface simple input/output devices with the 8085 in a simulation environment.

MODULE 4:

Types of memory: RAM, ROM, EPROM, EEPROM. Memory hierarchy: Registers, cache, main memory, secondary storage. Cache memory: Mapping techniques (direct, associative, set-associative), coherence protocols (MESI). Memory technologies: SRAM, DRAM, ROM. Memory optimization: Prefetching, write-through vs. write-back policies. Concept of virtual memory. CPU organization: Single-cycle and multi-cycle data path, control unit design (hardwired vs. microprogrammed).

Hands-On Activities:

1. Simulate cache mapping and replacement policies.
2. Analyze performance impact of memory hierarchy using tools like GEM5 or SimpleScalar.

MODULE 5:

Instruction-level parallelism: Superscalar processors, out-of-order execution. Branch prediction: Static and dynamic techniques, branch target buffers. SIMD and MIMD architectures: Introduction and applications. System interfacing: Memory-mapped vs. isolated I/O, DMA, interrupts (PIC, APIC). Modern interfaces: PCIe, USB, SATA, I^2C, SPI. Parallel architectures: Multi-core, SMP, NUMA, many-core systems. Interconnection networks: Bus, crossbar, Network-on-Chip (NoC).

Hands-On Activities:

1. Analyze multi-core performance using parallel processing benchmarks.
2. Simulate I/O interfacing scenarios.

MINOR PROJECTS DURING THE SEMESTER:

1. Design a Cache Simulator to evaluate hit/miss rates under various mapping and replacement policies.
2. 8085 Microprocessor-Based Traffic Light Controller simulation.
3. Benchmarking and Analyzing Multi-Core CPU Performance using parallel algorithms.
4. Simulation of a Virtual Memory Paging System with different replacement policies.

PEDAGOGY:

Mentor should introduce each topic through practical, real-world scenarios such as processor performance in gaming, IoT device interfacing, or multi-core server optimization. Students should approach problems using a "How Computers Really Work"

methodology, enabling them to derive hardware and software solutions naturally through experimentation rather than rote memorization.

RECOMMENDED TEXTBOOKS

Primary Textbooks:

1. M. Morris Mano and Charles R. Kime, Logic and Computer Design Fundamentals, Pearson.
2. David A. Patterson and John L. Hennessy, Computer Organization and Design, Morgan Kaufmann.
3. Supplementary Textbooks:
4. Ramesh S. Gaonkar, Microprocessor Architecture, Programming, and Applications with the 8085, Prentice Hall.
5. William Stallings, Computer Organization and Architecture: Designing for Performance, Pearson.
6. John L. Hennessy and David A. Patterson, Computer Architecture: A Quantitative Approach, Morgan Kaufmann.

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech (Artificial Intelligence & Mathematical Innovations)
Semester V
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-502

Course Title: The Logic Behind Machines: Theory of Computation

Credits: 06

Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

1. To apply mathematical foundations, algorithmic principles and computer science theory to the modeling and design of computational systems.
2. To demonstrate knowledge of basic mathematical models of computation and describe how they relate to formal languages.
3. To understand the limitations of computers and know about unsolvable problems.
4. To understand different phases, intermediate representations, algorithms and principles of working of a compiler

MODULE – I REGULAR EXPRESSIONS AND LANGUAGES:

Sets, relations, functions; strings, alphabets, and languages, Regular expressions and their algebra, Regular grammar and languages, Closure properties, Finite automata (FA), Mealy and Moore machines, Applications of regular expressions

Hands-On Activities:

i. Simulate and test regular expressions for pattern matching
ii. Convert regular expressions to equivalent finite automata

MODULE – II FINITE AUTOMATA

Non-Deterministic and Deterministic Finite Automata, Equivalence of Regular Expression and Finite automata, Equivalence of ε-NFA and NFA, Equivalence of NFA and DFA, Pumping Lemma for Regular Languages, Applications of finite automata.

Hands-On Activities:

i. Design DFAs for language recognition
ii. Validate input strings against NFAs and DFAs using simulation tools

MODULE – III CONTEXT FREE GRAMMAR

Grammar and its classification, Production rules and derivation, Context free Languages, Closure properties for context free languages, Pushdown Automata, Backus-Naur Form, Chomsky Normal Form, Griebach Normal Form, Pumping Lemma for Context free languages, Applications of Context Free Grammar.

Hands-On Activities:

i. Convert CFGs to CNF and GNF
ii. Design PDAs for balanced parentheses and palindromes

MODULE – IV TURING MACHINES

Description, Transition diagram, Roles of Turing machine, Church-Turing Thesis, Modular Construction of complex Turing machines, Extensions of Turing machines, Non-Deterministic Turing Machines. Universal Turing Machine, Turing acceptable and Turing decidable languages.

Hands-On Activities:

i. Design a TM for binary addition or palindrome checking
ii. Explore simulations of Universal Turing Machines

MODULE – V FUNCTION THEORY

Recursive Function Theory and Unsolvable Problems Partial, total and constant functions, Primitive recursive functions; Unbounded minimalization and μ-recursion; Decidable and Undecidable Problems, The Halting Problem, Reduction to Another Undecidable Problem, Undecidability of Post Correspondence Problem.

Hands-On Activities:

i. Analyze problems for decidability
ii. Discuss proof sketches for undecidability using reduction techniques

MINOR PROJECTS DURING THE SEMESTER:

i. Design a lexical analyzer using regular expressions and finite automata
ii. Construct a PDA for parsing a simple expression grammar
iii. Analyze the halting problem for different machine models

PEDAGOGY:

Mentor must introduce each topic by connecting it to real-life computing scenarios such as language design, compilers, and problem-solving in computer science. Concepts like automata, grammars, and Turing machines should be taught through visual tools, simulators, and relatable analogies to enable intuitive understanding. Students should be encouraged to discover patterns, construct machines, and explore computational

boundaries using the "How to Solve it" approach. Abstract theoretical ideas must unfold organically through engaging examples rather than as rigid formal definitions.

RECOMMENDED TEXTBOOKS:

1. H. R. Lewis and C. H. Papadimitriou - Elements of the Theory of Computation, Prentice Hall of India.
2. J. E. Hopcroft, R. Motwani and J. D Ullman - Introduction to Automata Theory, Languages and Computation, Pearson Education Asia.

3. Michael Sipser, Introduction to the Theory of Computation, Second Edition, Thomson, 2006.
4. Jeffrey Shallit, A Second Course in Formal Languages and Automata Theory, Cambridge University Press, 2008.
5. K. L. P. Mishra and N. Chandrasekaran - "Theory of Computations (Automata, languages and Computation)", Prentice Hall.
6. Rogers H., Theory of Recursive Functions and effective computing, Mcgraw-Hill
7. J.C.Martin–Introduction to Languages and Theory of Computation, Tata Mcgraw Hill.

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester V
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-503

Course Title: Information exchange in computing devices: Data Communication & Computer Networks

Credits: 06
Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

1. To study the basic taxonomy and terminology of the computer networking model and architecture.
2. To study the fundamentals of data communication and protocols.
3. To study network design and performance issues.
4. To explore the basic knowledge of cryptography and network security.

MODULE - I FUNDAMENTALS OF COMMUNICATION

Fundamentals of Communication, Modulation, Data Encoding, OSI reference model, TCP/IP model, network standardization, Inter-networking. Physical layer, Switching Technique, Transmission media, Co-axial, Twisted Pair and Fiber Optic Cables, Transmission Impairments, Electromagnetic Spectrum, Radio waves, Microwaves, Satellites, Wireless Mobile Telecommunications Technology.

Hands-On Activities:

i. Use spectrum analyzers to explore signal frequencies
ii. Set up basic wired and wireless transmission models

MODULE - II DATA TRANSMISSION AND MEDIA ACCESS METHODS

Data Link layer, Design issues, Frame, Error detection and correction, Flow Control, Elementary Data link protocols, Character-oriented and Bit-oriented Protocols, Sliding window protocols. Channel allocation methods, TDM, FDM, ALOHA, Carrier sense Multiple access protocols, Collision free protocols, IEEE standard 802 for LANS, Ethernet, Token Bus, Token ring.

Hands-On Activities:

i. Simulate sliding window and stop-and-wait protocols
ii. Implement CSMA/CD and compare with ALOHA in NS2/NS3

MODULE - III NETWORK ESTABLISHMENT CONCEPTS

Network Layer, Store and Forward Packet Switching, Connectionless and Connection-oriented services, Virtual Circuit, Routing Algorithms, Shortest path, Flooding, Link State, Distant vector, Hierarchical, Broadcast and Multicast Routing. OSPF, BGP, Congestion, Congestion control algorithms.

Hands-On Activities:

i. Visualize routing algorithms using simulation tools
ii. Analyze congestion using network traffic simulators

MODULE – IV INTERNET PROTOCOLS

TCP/TP Protocol, IP Addresses, Classes of IP Addresses, Subnets, IPv6, Network layer in the Internet, Internet Control Protocols, ARP, RARP, BOOTP, DHCP, Transport Layer, Protocol Stack, TCP and UDP, Transport Services Primitives, Sockets, Socket Programming concept.

Hands-On Activities:

i. Develop basic TCP and UDP socket programs in Python
ii. Configure IP addressing and subnetting on local networks

MODULE – V NETWORK APPLICATION AND NETWORK SECURITY

Application layer, Name service (DNS), Domain Hierarchy, Name servers, Name resolutions, Traditional applications, Telnet, FTP, SMTP, MIME, World wide web-HTTP, HTTP Methods. Cryptographic Algorithms, DES, AES, RSA, Key exchange methods, Authentication Protocol, Digital Signatures.

Hands-On Activities:

i. Simulate secure message exchange using RSA
ii. Set up DNS and FTP servers in a virtual lab environment

MINOR PROJECTS DURING THE SEMESTER:

i. Design a secure file transfer protocol using TCP and encryption
ii. Analyze routing efficiency in dynamic topologies
iii. Implement DNS resolver with local caching

PEDAGOGY:

Mentor must introduce each topic by relating it to everyday technologies like email, web browsing, mobile communication, and IoT devices to foster intuitive understanding. Networking concepts such as protocols, transmission methods, and addressing should emerge naturally through real-life problem scenarios like online video streaming or secure file transfer. Emphasis should be laid on simulation-based learning and hands-on lab experiments using packet tracers, socket programming, or protocol analyzers. Students

should be guided to build mental models of layered architectures and data flows by following a "How to Solve it" approach, empowering them to design, troubleshoot, and optimize networks.

RECOMMENDED TEXTBOOKS:

1. Andrew S. Tanenbaum, "*Computer Networks*", 5 e, 2013, Pearson Education Asia.
2. Behrouz A. Forouzan, "*Data Communications and Networking*", 4e, 2004, Tata McGraw Hills.
3. William Stallings. "*Data and Computer Communication*", 7e, 2016, Pearson Education Asia.
4. Prakash C. Gupta, "*Data Communications and Computer Networks*", PHI
5. Michael A. Miller, "*Data and Network Communications*", 2e, Delmar Thomson Learning.
6. James F. Kurose and Keith W. Ross, "*Computer Networking*", 3e, Pearson Education.
7. William A. Shay, "*Understanding Data Communications and Networks*", 2e, Thomson Asia Pvt. Ltd.

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester V
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-504

Course Title: Instructing computing devices: Operating System

Credits: 06

Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

1. To learn the fundamentals of Operating Systems.
2. To learn the mechanisms of OS to handle processes and their communication.
3. To learn the mechanisms involved in memory management in OS.
4. To brief the students about basic concepts of Unix & Linux and programs using shell programming.

MODULE – I INTRODUCTION TO OPERATING SYSTEMS

Evolution of operating systems, Operating systems concepts, Types of operating systems, Different views of the operating system, Operating system services, System calls, Types of system calls. Operating system Structure, Layered Approach, Microkernels, Virtual machines.

Hands-On Activities:

i. Use system calls like fork, exec, wait in a Linux environment
ii. Identify OS structures in various Linux distributions

MODULE - II PROCESS MANAGEMENT

Process concept, Operation on processes, Inter-process communication, Mutual exclusion, Introduction to Process scheduling, Scheduling algorithms, Process Synchronization, Inter process Synchronization, Critical section problem, Semaphores, Monitors, Message passing. Deadlocks, System Model, Deadlock characterization, Deadlock prevention, Deadlock avoidance.

Hands-On Activities:

i. Simulate process scheduling algorithms
ii. Implement producer-consumer problem using semaphores in C/Linux

MODULE - III MEMORY MANAGEMENT

Memory management, Swapping, Contiguous memory allocation, Relocation & protection, Memory management, Paging, Segmentation, Intel Pentium Segmentation, Intel Pentium Paging, Virtual memory, Demand paging, Performance of demand paging, Page replacement algorithms: FIFO, Optimal, LRU, Counting based page replacement.

Hands-On Activities:

i. Analyze page faults using simulation tools
ii. Write code to simulate LRU and FIFO algorithms

MODULE - IV FILE & I/O MANAGEMENT

File & I/O Management Files system structure, File system implementation, Directory Implementation. Allocation Methods, contiguous allocation, linked allocation, Indexed allocation Disk organization, Disk space management, Disk scheduling, Disk Management, RAID Structure.

Hands-On Activities:

i. Simulate file allocation techniques
ii. Analyze disk scheduling with real input-output logs

MODULE – V INTRODUCTION TO LINUX/UNIX

Directory tree, file types, file permissions, Common UNIX commands: ls, cat, mv, cp, grep, chmod, etc., Shell programming basics: Variables, control structures, loops, Filters and text-processing tools, Introduction to VI editor

Hands-On Activities:

i. Write shell scripts for batch file operations and log processing
ii. Explore directory and permission structures in Linux

MINOR PROJECTS DURING THE SEMESTER:

i. Simulate a simple shell with process and file management commands
ii. Design and analyze scheduling algorithms using real or synthetic data
iii. Develop a memory management visualizer for paging and segmentation

PEDAGOGY

Mentor must introduce operating system concepts through relatable, real-world computing experiences such as multitasking on smartphones, file management, and memory usage in common applications. Abstract concepts like process scheduling, memory management, and system calls should be demonstrated using simulations and practical examples within Linux/UNIX environments. Students should be encouraged to experiment through shell scripting, process tracing, and memory allocation exercises. Teaching should follow the "How to Solve it" approach, enabling learners to internalize

the design and functioning of an OS by building intuition and problem-solving skills, rather than memorizing definitions and mechanisms.

RECOMMENDED TEXTBOOKS:

1. Silberschart, Galvin, Gagne, "Operating System Concepts", 9th Edition, WSE Wiley, 2016.

2. Andrew. S. Tanenbaum, "Modern operating systems", 4th Edition, Pearson Prentice Hall, 2018

3. Milan Milenkovic, "Operating system-concepts and design", 2nd Edition, McGraw Hill International Edition, 2005

4. A. S. Godbole, "Operating systems", 3rd Edition, Tata McGraw hill, 2017.

5. Deitel H. M., "Operating System", 3rd Edition, Pearson Publications, 2012.

6. Madnick& Donovan, "Operating Systems", Tata McGraw Hill, 2003.

7. Sumitabha Das, "UNIX Concepts and Application", 4th Edition, Tata McGraw Hill, 2017.

8. Richard L. Petersen, "The Complete Reference Linux", 6th Edition, Tata McGraw Hill, 2010

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester V
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-505

Course Title: Building Blocks of Computing: Groups, Graphs, and Trees

Credits: 06

Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

i. Identify and verify the properties of groups and subgroups, and apply them to cryptographic principles.
ii. Represent real-world systems as graphs, analyze connectivity and related properties, and test for isomorphism.
iii. Apply algorithms such as Dijkstra's for shortest paths and use graph coloring techniques for scheduling and resource allocation.
iv. Understand tree properties and traversal methods, and apply Huffman coding for efficient data compression.
v. Implement Prim's and Kruskal's algorithms to find minimum spanning trees for cost-effective network design

MODULE – I INTRODUCTION TO ALGEBRAIC STRUCTURES & GROUPS

- Building Blocks of Operations: Discover how sets and binary operations form the backbone of mathematical structures used in computing.
- Group Power: Step into the world of groups—sets where closure, associativity, identity, and inverses rule the game.
- Abelian Harmony: Experience the simplicity of commutative groups and their role in symmetric systems.
- Cyclic Spins: See how cyclic groups revolve around a single generator, powering encryption and coding.
- Permutation Playgrounds: Explore permutation groups and their magic in rearranging data without losing structure.
- Subgroup Secrets: Identify subgroups and understand how smaller structures inherit group power.
- From Math to Machines: Apply group theory to cryptography, error detection, network security, and pattern recognition.

Hands-On Activities:

i. Operation Lab: Code addition modulo n and string concatenation; verify closure, associativity, identity, and inverse properties.
ii. Group Detective: Given a set and operation table, check if it forms a group; identify which property fails if it doesn't.
iii. Cyclic Explorer: Generate all elements of a cyclic group from one generator and visualize the cycle.
iv. Permutation Playground: List all permutations of a set and verify group properties under composition.
v. Crypto Mini-Challenge: Implement a simple RSA-like encryption using modular arithmetic and cyclic group concepts.

MODULE - II THE ART OF GRAPHS—CONNECTIONS UNVEILED

- Webs and Maps: Explore what graphs are and how we represent them (adjacency, incidence).
- Walking the Bridges: Discover Euler's Relation and tackle Eulerian paths.
- Spot the Twins: Learn how to recognize when two graphs are truly the same (isomorphism).
- Testing Strength: Identify points of fragility—connectivity, cut vertices, and bridges.
- Strategic Watching: Get the basics of covering sets.
- Epic Journeys: Seek out Euler and Hamilton paths and circuits.

Hands-On Activities:

i. Map a real network (classroom, bus routes) as a graph.
ii. Solve your own "Bridge of Königsberg" puzzle.
iii. Detect isomorphism between pairs of graphs.
iv. Remove nodes/edges to test network resilience.
v. Find Euler/Hamilton paths in sample graphs.

MODULE - III MEMORY MANAGEMENT

- Shortest Routes: Master Dijkstra's algorithm to find the quickest path through a network.
- The Salesman's Quest: Dive into the Travelling Salesman Problem, exploring challenges in finding optimal tours.
- Scheduling Success: Translate jobs and tasks into graphs to solve scheduling challenges efficiently.
- Matchmaking and Independence: Understand graph matching and independent sets to pair up elements or find conflict-free groups.
- Color Your World: Apply graph coloring strategies to solve real-world problems like resource allocation and scheduling.
- Planar Insights: Uncover the beauty of planar graphs and grasp Euler's formula's elegant balance.
- Hidden Obstacles: Explore Kuratowski's theorem to recognize graphs that can't be drawn on a plane without crossings.

Hands-On Activities:

i. Implement Dijkstra's algorithm on sample maps to find shortest paths.
ii. Experiment with small Travelling Salesman instances to understand complexity.
iii. Create task graphs and develop job schedules respecting dependencies.
iv. Solve simple matching problems and identify independent sets in graphs.
v. Apply graph coloring to timetable or resource allocation scenarios.
vi. Visualize planar graphs and verify Euler's formula.
vii. Detect non-planar graphs using Kuratowski's criteria.

MODULE - IV FILE & I/O MANAGEMENT

- Roots and Branches: Understand what trees are—their definitions and key properties shaping hierarchical structures.
- Walking the Tree: Explore tree traversal methods—preorder, inorder, and postorder—and see how they reveal different views of data.
- Secret Codes: Dive into prefix codes and Huffman coding, learning how trees help compress data efficiently and uniquely.
- Bridging Networks: Learn what spanning trees are and why they form the backbone of connecting all nodes without cycles.
- Building Minimal Links: Master Prim's and Kruskal's algorithms to craft minimum spanning trees, finding the least-cost way to connect everything.

Hands-On Activities:

i. Perform preorder, inorder, and postorder traversals on sample trees.
ii. Build a Huffman tree from character frequencies and encode/decode messages.
iii. Identify spanning trees in sample graphs.
iv. Implement Prim's and Kruskal's algorithms to find minimum spanning trees on weighted graphs.

PEDAGOGY

Each topic in this course begins with relatable real-life scenarios or practical problems, providing students with an intuitive grasp of core concepts. Emphasizing a "learning by doing" philosophy, students actively build their mathematical understanding through hands-on activities, interactive coding simulations, and collaborative classroom problem-solving. This approach ensures theory is naturally connected to application, fostering deeper insight and confidence in the subject.

RECOMMENDED TEXTBOOKS:

i. Gallian, J. A., Contemporary Abstract Algebra (10th ed.), Cengage Learning, 2018.
ii. Rosen, Kenneth H., Discrete Mathematics and Its Applications, 8th ed., McGraw-Hill Education, New York, NY, 2019.
iii. Epp, S. S., Discrete Mathematics with Applications (5th ed.), Cengage Learning, 2019.
iv. Liu, C. L., & Mohapatra, D. P., Elements of Discrete Mathematics: A Computer-Oriented Approach (3rd ed.), Tata McGraw Hill, 2008.

v. West, Douglas B. Introduction to Graph Theory, 2nd ed., Prentice Hall, Upper Saddle River, NJ, 2000.

vi. Software and Tools:
    a. Python (networkx, matplotlib, itertools)
    b. Gephi or Graphviz (for visualization)
    c. Spreadsheet tools like Excel for combinatorics
    d. C++ or Java for algorithm implementation

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester VI
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-601
Credits: 06
Contact Hours: 12 per Credit

Course Title: Linear Logic: The Art of Optimization
Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES: The objectives of the course on "Linear Logic: The Art of Optimization" include:

1. formulation and solution of linear programming problems using graphical and simplex methods;
2. solution of real-life problems using transportation and assignment models;
3. developing a solid understanding of game theory;
4. application of computational tools like MATLAB and GeoGebra to model and solve optimization problems.

MODULE - I CONCEPT OF OPTIMIZATION, LINEAR PROGRAMMING

Introduction, Formulation of a Linear Programming Problem (LPP), Requirements for an LPP, Advantages and limitations of LP. Graphical solution: Multiple, unbounded, and infeasible solutions.

Hands-On Activities:

i. Graphical solution of LPP using GeoGebra
ii. Identifying feasible region and optimal solution using GeoGebra

MODULE - II PRINCIPLE OF THE SIMPLEX METHOD

Standard form, basic solution, basic feasible solution. Computational Aspect of Simplex Method: Cases of unique feasible solution, no feasible solution, multiple solutions, unbounded solution, and degeneracy. Two Phase and Big-M methods. Duality in LPP, primal-dual relationship.

Hands-On Activities:

i. Implementing Simplex method using MATLAB

ii. Model motion planning problems when dynamics are linear (e.g., minimum-energy trajectories).

MODULE -–III LOGIC PROBLEMS

Unbalanced and degenerate transportation problems, transshipment problems, and maximization in a transportation problem. Assignment Problem: Solution by the Hungarian method, Unbalanced assignment problem, Maximization in an assignment problem, Crew assignment, and Travelling salesman problem

Hands-On Activities:

i. MODI method for optimal transportation solution using MATLAB
ii. Maximization in Assignment: Matchmaking, recommendation engines, strategic resource use.

MODULE -–IV GAME THEORY

Two-person zero sum games, maxmin-minmax principle, games without saddle points (Mixed strategies), graphical solution of 2×n and m×2 games, dominance property, arithmetic method of n× n games, general solution of m × n rectangular games.

Hands-On Activities:

i. Use graphing tools (GeoGebra or Desmos) to solve 2×n and m×2 games.
ii. Application in real-life scenarios (e.g., cybersecurity, bidding wars).

MINOR PROJECTS DURING THE SEMESTER:

i. Optimizing Food Distribution for a Campus Canteen Using LPP and Transportation Models
ii. Efficient Ride Allocation for a Campus Shuttle Service Using Assignment and Game Theory
iii. Designing Minimum-Cost Marketing Strategies for Local Startups Using Simplex and Duality

PEDAGOGY:

Mentor must introduce each topic with the help of real-life situations/problems so as to give complete understanding of the concept and enabling the students to find solutions to the problems at their own by "How to Solve it" approach. Mathematical concepts must come to the students in a natural way instead of imposing on them.

RECOMMENDED TEXTBOOKS

Primary Textbooks:

1. H. A. Taha, Operations Research-An Introduction, Macmillan Publishing Company Inc., 2006.

2. A. Ravindran, D.T. Phillips, and J.J. Solberg, Operations Research: Principles and Practice, 2nd Edition, John Wiley and Sons, 1987.

Supplementary Textbooks:

1. F.S. Hiller, and G.J. Liebermann, Introduction to Operations Research, Tata McGraw Hill, 2000.
2. K. Swarup, P.K. Gupta, and M. Mohan, Operations Research, Sultan Chand & Sons, New Delhi, 2001.

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech (Artificial Intelligence & Mathematical Innovations)
Semester VI
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-602

Course Title: Exploring Database Management Systems

Credits: 06

Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

1. Understand DBMS architecture and operations
2. Design ER-models and transform them into schemas
3. Write and optimize SQL queries

MODULE –I DATABASE CONCEPTS

File-based vs DBMS, architecture, Schemas, data independence, centralized & client-server DBMS,

Hands-On Activities:

1. Create sample databases using MySQL/PostgreSQL
2. Simulate database file structures

MODULE - II RELATIONAL DATA MODEL

ER to relational mapping, relational algebra/calculus, Joins and queries using relational models

Hands-On Activities:

1. Build ER diagrams using draw.io or MySQL Workbench
2. Query practice with JOINs and nested queries.

MODULE – III NORMALIZATION

Functional dependencies, keys, 1NF to 5NF, BCNF, MVDs

Hands-On Activities:

1. Normalize unstructured tables
2. Design fully normalized schema in DBMS

MODULE - IV CONCURRENCY CONTROL

Transactions, deadlocks, concurrency, Locking, timestamp ordering, recovery

Hands-On Activities:

1. Simulate concurrent transactions
2. Implement two-phase locking in SQL scripts

MODULE – V SQL

SQL syntax, joins, views, inbuilt functions, Roles, privileges, data integrity

Hands-On Activities:

1. SQL programming challenges on HackerRank or DB Fiddle
2. Build a secure, role-based SQL database

MINOR PROJECTS DURING THE SEMESTER:

1. Student Record Management System
2. Role-based Hospital DBMS
3. E-commerce Order Tracking DB

PEDAGOGY:

Mentor must introduce each database concept by linking it to real-life applications such as library management systems, hospital records, e-commerce platforms, or social media analytics. Concepts like ER modeling, relational design, normalization, SQL querying, and transaction management should be taught through a balance of theory and hands-on implementation using industry-standard tools like MySQL or PostgreSQL. Students should compare different database models and query languages to understand trade-offs in design and performance. Emphasis should be placed on visualization of schemas, query execution plans, and concurrency scenarios. Through the "How to Solve it" approach, learners should be encouraged to identify the most appropriate database structures and operations for a given problem, focusing on efficient, secure, and scalable solutions rather than rote query writing.

RECOMMENDED TEXTBOOKS:

1. Bipin C. Desai, An Introduction to Database Systems
2. Ramez Elmasri & Shamkant B. Navathe, Fundamentals of Database Systems
3. C. J. Date, An Introduction to Database Systems

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester VI
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-603

Course Title: Architecting the Web: Modern Web Application Engineering

Credits: 06
Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

1. Understand web architecture, frontend/backend integration, and security practices.
2. Build full-stack, responsive web applications using modern technologies.
3. Use DevOps, CI/CD, and cloud deployment in real-world projects.
4. Explore modern web architectures like PWAs, Microservices, Serverless, and GraphQL.

MODULE – I FOUNDATIONS OF WEB ARCHITECTURE AND FRONTEND BASICS

Evolution: Static to SPA to PWA, Client-server model, HTTP/HTTPS, DNS, HTML5, CSS3, JavaScript (ES6+), DOM, async programming, Responsive design: Flexbox, Grid, media queries, Tooling: npm, Webpack/Vite, Babel, Git

Hands-On Activities:

1. Build a responsive webpage using HTML, CSS, and JS
2. DOM manipulation and async interaction using Promises
3. Set up a project with npm and Webpack

MODULE – II WEB TECHNOLOGIES, APIS, AND BACKEND FUNDAMENTALS

Frontend: React/Vue/Angular; Backend: Node.js, PHP, Flask, REST API: CRUD, JWT, OAuth 2.0, SQL vs NoSQL; ORM tools, SSR vs CSR, Microservices overview, API docs: Swagger/OpenAPI

Hands-On Activities:

1. Create a RESTful API using Express.js or Flask
2. Integrate MongoDB or PostgreSQL
3. API authentication using JWT

MODULE – III FULL-STACK DEVELOPMENT AND STATE MANAGEMENT

Component design: Virtual DOM, props, state, State management: Redux, Context API, Routing and SPA patterns, Full-stack frameworks: Next.js, GraphQL basics, WebSockets, real-time messaging.

Hands-On Activities:

1. Build a small React+Node full-stack app
2. Implement WebSocket-based chat feature
3. Use Apollo for GraphQL integration

MODULE – IV PERFORMANCE, SECURITY, AND DEVOPS PRACTICES

Performance: lazy loading, CDN, caching, Security: XSS, CSRF, input validation, CORS, Testing: Jest, Cypress, CI/CD: GitHub Actions, Docker, Logging & Monitoring: ELK stack, Prometheus

Hands-On Activities:

1. Create CI/CD pipeline with GitHub Actions
2. Dockerize a web app
3. Write unit & integration tests with Jest

MODULE – V ADVANCED ARCHITECTURES AND EMERGING TRENDS

Microservices, serverless (AWS Lambda, Firebase), Scalability: Load balancing, autoscaling, Terraform, PWAs, Edge computing, Jamstack, WASM, AI/ML integration, Kafka, Ethics: Privacy, accessibility, sustainability

Hands-On Activities:

1. Convert app to PWA using service workers
2. Deploy microservice to cloud (e.g., Vercel or Firebase)
3. WASM demo using Rust or C++ modules in JS

MINOR PROJECTS DURING THE SEMESTER:

1. Build an e-commerce full-stack PWA
2. Design a scalable chat app using microservices and GraphQL
3. Develop a dashboard with Docker-based deployment and monitoring
4. Create a serverless blog with authentication and GraphQL API

PEDAGOGY:

Mentor must introduce each web development concept by anchoring it to real-world scenarios such as designing an e-commerce portal, a social networking site, or a live data dashboard. Frontend and backend topics should be taught through progressive, hands-on projects. Emphasis should be placed on version control, collaborative coding, performance optimization, and deployment practices. Through the "How to Solve it" approach.

RECOMMENDED TEXTBOOKS:

1. Stoica & Tanenbaum, Distributed Systems: Principles and Paradigms
2. MacDonald, Pro ASP.NET Core 3
3. Martin Fowler, Patterns of Enterprise Application Architecture
4. Mozilla MDN: https://developer.mozilla.org
5. Google Web Fundamentals: https://web.dev

UNIVERSITY OF JAMMU

Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester VI
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-604

Credits: 06
Contact Hours: 12 per Credit

Course Title: Computer Graphics and Visualization using Java
Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

LEARNING OUTCOMES:

1. Understand fundamental concepts of computer graphics and visualization.
2. Learn 2D and 3D graphics programming using Java.
3. Implement foundational graphics algorithms and transformations.
4. Build interactive Java-based visual applications.
5. Apply visualization techniques for rendering, clipping, and animation.

MODULE - I FUNDAMENTALS OF COMPUTER GRAPHICS

Introduction to Computer Graphics and its applications, Raster vs Vector graphics, Color models: RGB, CMYK, HSV, Java 2D API and Graphics2D class, Drawing shapes (lines, ovals, rectangles), Raster graphics concepts: Pixels, resolution, frame buffers, Basic scan conversion algorithms for line and circle drawing

Hands-On Activities:

1. Drawing shapes using Java 2D API (drawLine, drawRect, drawOval)
2. Color manipulation using RGB models
3. Implement and visualize DDA and Bresenham algorithms in Java

MODULE - II GRAPHIC PRIMITIVES

Concept of graphic primitives: points, lines, circles, ellipses, Algorithms: DDA, Bresenham (line/circle), Midpoint ellipse, Area filling: Boundary fill, Flood fill, Scan-line fill, Aliasing and filtering: Sampling, halftoning, anti-aliasing techniques

Hands-On Activities:

1. Java implementation of area filling algorithms
2. Visual demonstration of aliasing and anti-aliasing
3. Real-time shape drawing and user interaction

MODULE - III GEOMETRIC TRANSFORMATIONS

2D and 3D transformations: Translation, Scaling, Rotation, Shearing, Reflection, Homogeneous coordinate system, Matrix representation and composite transformations

Hands-On Activities:

1. Implement 2D transformations with GUI controls
2. Animate geometric objects undergoing multiple transformations
3. Visualize composite transformations using Java Graphics2D

MODULE - IV VIEWING & CLIPPING TRANSFORMATIONS

Projections: Parallel, Orthographic, Oblique, Isometric, Perspective projections: Vanishing points, 1-point, 2-point, 3-point, Clipping techniques: Cohen–Sutherland, Cyrus–Beck

Hands-On Activities:

1. Visualize various projections using Java 3D
2. Implement Cohen–Sutherland line clipping
3. Create a simple viewport to clip and render a 3D object

MODULE - V THREE-DIMENSIONAL OBJECT REPRESENTATION

Polygon meshes and plane equations, Curves and surfaces: Bezier curves, Hermite interpolation, Hidden surface removal: Z-buffer, scan-line, Painter's algorithm, Shading models: Flat, Gouraud, Phong

Hands-On Activities:

1. Render Bezier curves and control points
2. Simulate 3D object rotation with shading
3. Implement Z-buffer-based hidden surface removal in Java

MINOR PROJECTS DURING THE SEMESTER:

1. Interactive Paint Application using Java 2D and raster graphics algorithms
2. 3D Wireframe Visualizer with projection and transformation support
3. Graph Coloring Simulator with user input and visual feedback
4. Bezier Curve Editor with real-time rendering and control-point manipulation
5. Polygon Clipping Tool for visualizing line and area clipping in 2D/3D scenes

PEDAGOGY:

Mentor must link each computer graphics concept to real-world uses such as drawing tools, interactive dashboards, or simple games. Topics like coordinate systems, transformations, and animation should be taught through progressive Java-based projects using AWT, Swing, and JavaFX. Visualization should involve relatable datasets to make concepts tangible. The "How to Solve it" approach should guide problem breakdown, coding, debugging, and performance tuning, with emphasis on collaboration and version control.

RECOMMENDED TEXTBOOKS:

1. Foley, J.D., et al., Computer Graphics: Principles and Practice, Addison-Wesley
2. Hearn, D. & Baker, M.P., Computer Graphics with OpenGL, Pearson Education
3. Hill, F.S., Computer Graphics Using OpenGL, Pearson Education
4. Rogers, D.F., Procedural Elements for Computer Graphics, McGraw-Hill
5. Watt, A., 3D Computer Graphics, Addison-Wesley

UNIVERSITY OF JAMMU
Four Year Innovative Undergraduate Program
(Design Your Degree)
B. Tech. (Artificial Intelligence & Mathematical Innovations)
Semester VI
(For the session 2025, 2026, 2027)

Course Code: UMJDTT-605

Course Title: Data Structure Design Using Object-Oriented Programming C\C++

Credits: 06
Contact Hours: 12 per Credit

Maximum Marks : 150
Mid Semester Exam : 30
End Semester Exam : 70
Project Cumulative : 50

Learning outcomes:

1. Understand and apply fundamental data structures using C and C++ programming.
2. Implement object-oriented concepts for designing modular and reusable data structures.
3. Compare C-based and C++ class-based design methodologies.
4. Solve computational problems using linear and non-linear data structures.
5. Develop mini-projects integrating custom data structures with real-world use cases.

MODULE - I FOUNDATIONS OF DATA STRUCTURE DESIGN WITH C AND C++

C basics: structs, pointers, dynamic memory (malloc/free), C++ basics: classes, objects, constructors/destructors, designing data structures: arrays and structs in C vs. classes in C++, Memory management: dynamic allocation in C vs. C++

Hands-On Activities:

i. Implement dynamic arrays in both C and C++
ii. Compare memory leaks and management using malloc/free vs. new/delete

MODULE -–II OBJECT-ORIENTED PROGRAMMING FOR DATA STRUCTURES

Core OOP principles: encapsulation, inheritance, polymorphism, Designing ADTs using C++ classes, Access specifiers and class hierarchies, Struct-based vs. class-based design comparisons

Hands-On Activities:

i. Create stack and queue as ADTs in C++
ii. Demonstrate function and operator overloading for data types

MODULE - III DESIGNING LINEAR DATA STRUCTURES

Linked lists: singly and doubly linked lists, Stack: array-based and linked-list-based implementations, Queue: linear, circular, and linked-list-based, OOP design: encapsulation, polymorphism in stack/queue operations

Hands-On Activities:

i. Build linked lists using structs and classes
ii. Compare performance and use-cases of different queue implementations

MODULE - IV DESIGNING NON-LINEAR DATA STRUCTURES

Binary trees and binary search trees (BSTs), Graph representations: adjacency matrix vs. list, Heaps: min-heap and max-heap as priority queues, Use of inheritance and polymorphism in tree and graph designs

Hands-On Activities:

i. Implement tree traversals (inorder, preorder, postorder)
ii. Simulate BFS/DFS using graph classes in C++
iii. Use heaps for task scheduling

MODULE - V REAL-WORLD DATA STRUCTURE APPLICATIONS

Design patterns in DS: Iterator, Strategy, Composite, Sorting and searching algorithms using OOP, Introduction to Standard Template Library (STL), Boost, modern C++ features

Mini-project planning: social network, scheduler, etc.

Hands-On Activities:

i. Implement sort/search functions with performance comparison
ii. Use STL for advanced DS design and manipulation

MINOR PROJECTS DURING THE SEMESTER:

1. Build a custom task scheduler with priority queues and linked lists
2. Create a graph-based social network explorer
3. Develop an STL-enhanced application like a mini-database manager

PEDAGOGY:

Mentor must introduce each data structure by anchoring it to real-life problem contexts such as task scheduling, social networks, or file systems. Concepts like stacks, queues, trees, and graphs should be taught using both C and C++ implementations, allowing students to compare procedural and object-oriented design approaches. Emphasis should be placed on visualization, modular coding, and hands-on practice to strengthen problem-solving abilities. Through the "How to Solve it" approach, students should be encouraged to identify the most suitable data structure for a given scenario and design efficient, reusable code structures naturally—rather than learning them in isolation.

RECOMMENDED TEXTBOOKS:

1. Yashavant Kanetkar, Let Us C, BPB Publications (2016)
2. E. Balagurusamy, Object-Oriented Programming with C++, McGraw Hill (2007)
3. Supplementary Textbooks:
4. Michael T. Goodrich, R. Tamassia, Data Structures and Algorithms in C++, Wiley (2014)
5. Y. Langsam, M. Augenstein, A. Tenenbaum, Data Structures Using C and C++, Prentice Hall
6. Mark Allen Weiss, Data Structures and Algorithm Analysis in C++, Pearson Education